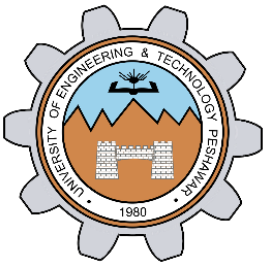
UNIVERSITY OF ENGINEERING AND TECHNOLOGY, PESHAWAR

LAB REPORT

Application of Mask R-CNN for Car Damage Detection and Instance Segmentation



Artificial Neural Networks & Deep Learning Lab

Fall 2026 | 5th Semester

Submitted by

Muhammad Arsalan

Registration No: 24PWAI0015 | Section: AI

Submitted to

Lecturer Hasib Shamshad

"On my honor, as a student of the University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work."

Student Signature: _____

Bachelor of Science in Artificial Intelligence

1. Introduction

Automated damage assessment supports insurance claims, used-vehicle inspection and repair estimation. Image classification labels a whole photo and bounding-box detectors (Faster R-CNN, YOLO) only localise damage coarsely, yet repair cost depends on the exact shape and area of the damage. Instance segmentation provides that pixel-accurate boundary.

This lab applies **Mask R-CNN** with a ResNet-101 + FPN backbone, fine-tuned from MS COCO weights on **68 annotated car-damage photographs**, to segment scratches and quantify their area from the predicted polygons.

2. Mask R-CNN Architecture

Mask R-CNN (He et al., ICCV 2017) extends the two-stage Faster R-CNN detector with a third, parallel branch that predicts a segmentation mask for every detected region.

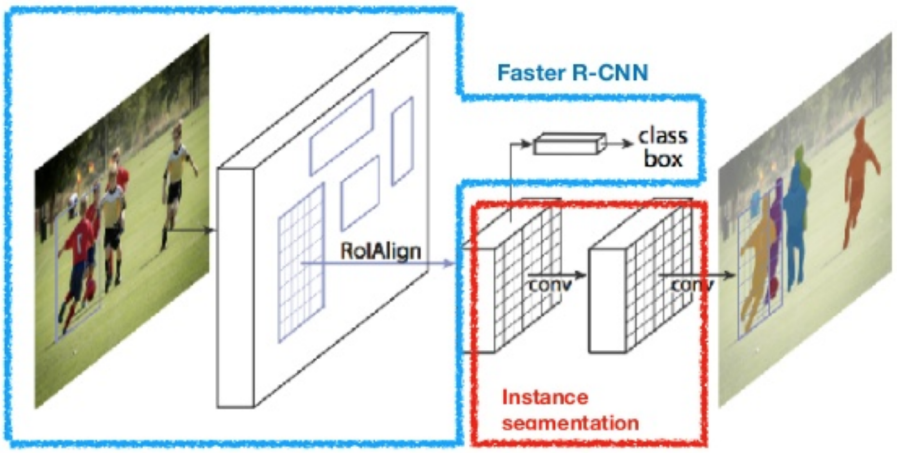


Figure 1. Mask R-CNN pipeline: Faster R-CNN with a parallel fully convolutional mask branch.

Component	Role
Backbone	ResNet-101 with a Feature Pyramid Network (P2–P6): bottom-up and top-down paths joined by 1×1 lateral convolutions, so both hairline scratches and large dents are detected.
RPN	Slides over each feature map with anchors of 32–512 px and ratios 1:2, 1:1, 2:1; outputs objectness scores and box deltas. After NMS (0.7) it keeps 2,000 RoIs for training and 1,000 for inference.
RoIAlign	Replaces RoIPool's coordinate rounding with bilinear interpolation, giving the sub-pixel alignment that fine scratch masks need.
Heads	Parallel classification, box-regression and FCN mask branches. The mask branch predicts an m×m binary mask per class, which avoids competition between classes.
Loss	$L_{total} = L_{cls} + L_{bbox} + L_{mask}$ (cross-entropy, smooth-L1 and per-pixel binary cross-entropy).

Damage area. For a mask polygon with vertices $(x_1, y_1) \dots (x_n, y_n)$, the area follows from Gauss's Shoelace formula:

$$A = \tfrac{1}{2} \left| \Sigma_{i=1..n} \left(x_i \, y_{i+1} - x_{i+1} \, y_i \right) \right|, \text{ with } (x_{n+1}, y_{n+1}) = (x_1, y_1)$$

The pixel area can be converted to cm^2 using a reference object for calibration.

3. Dataset and Annotation

The photographs cover different car models, colours, lighting conditions, viewpoints and damage severity. Each damaged area was outlined as a polygon with the VGG Image Annotator and exported to `via_region_data.json`.

Total images	Training	Validation	Test (unseen)	Annotation	Class
68	50	16	2	VIA 1.0.6 (polygons, JSON)	scratch

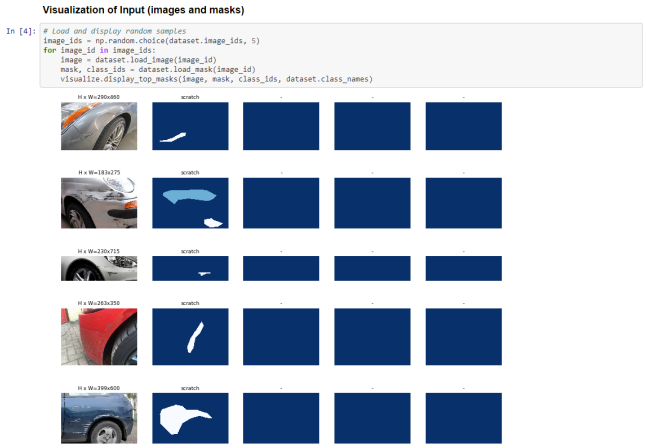


Figure 2. Ground-truth scratch masks on randomly sampled training images.

```python
class CustomDataset(utils.Dataset):
    def load_custom(self, dataset_dir, subset):
        """Load a subset of the dataset.
        dataset_dir: Root directory of the dataset.
        subset: Subset to load: train or val
        """
        # Add classes. We have only one class to add.
        self.add_class("scratch", 1, "scratch")

        # Train or validation dataset?
        assert subset in ["train", "val"]
        dataset_dir = os.path.join(dataset_dir + "dataset/" + subset)
```

Figure 3. Dataset loader (load_custom in custom.py).

4. Transfer Learning and Training

Training a 101-layer network from scratch on 68 images would overfit, so the model is initialised from MS COCO weights (`mask_rcnn_coco.h5`). The class-specific output layers are re-initialised for two classes (background and scratch), and training uses `layers='heads'` for 15 epochs with SGD while the ResNet-101 backbone stays frozen.

Parameter	Value	Parameter	Value
`BACKBONE`	resnet101	`NUM_CLASSES`	2 (BG + scratch)
`IMAGE_MIN / MAX_DIM`	800 / 1024	`BACKBONE_STRIDES`	[4, 8, 16, 32, 64]
`RPN_ANCHOR_SCALES`	(32, 64, 128, 256, 512)	`RPN_ANCHOR_RATIOS`	[0.5, 1, 2]
`LEARNING_RATE`	0.001	`LEARNING_MOMENTUM`	0.9
`STEPS_PER_EPOCH`	100	`DETECTION_MIN_CONFIDENCE`	0.90
`EPOCHS (heads)`	15	`LOSS_WEIGHTS`	1.0 for all five losses

```python
def train(model):
    """Train the model."""
    # Training dataset.
    dataset_train = CustomDataset()
    dataset_train.load_custom(args.dataset, "train")
    dataset_train.prepare()

    # Validation dataset
    dataset_val = CustomDataset()
    dataset_val.load_custom(args.dataset, "val")
    dataset_val.prepare()

    # *** This training schedule is an example. Update to your needs ***
    # Since we're using a very small dataset, and starting from
    # COCO trained weights, we don't need to train too long. Also,
    # no need to train all layers, just the heads should do it.
    print("Training network heads")
    model.train(dataset_train,dataset_val,
                learning_rate=config.LEARNING_RATE,
                epochs=15,
                layers='heads')
```

Figure 4. Training routine (train in custom.py).

5. Weight Diagnostics

Weights and biases of the trainable Conv2D, Dense and Conv2DTranspose layers were inspected to check numerical stability. Kernels follow a smooth distribution centred near zero (conv1 kernel: mean ≈ 0.00, std ≈ 0.13, range −0.86 to +0.84) and biases start at zero, so there is no sign of exploding or vanishing weights.

Perform a sanity check if weights and biases are properly distributed.

In [12]:
```python
# layer types to display
LAYER_TYPES = ['Conv2D', 'Dense', 'Conv2DTranspose']

# Get layers
layers = model.get_trainable_layers()
layers = list(filter(lambda l: l.__class__.__name__ in LAYER_TYPES,
                     layers))
# Display Histograms
fig, ax = plt.subplots(len(layers), 2, figsize=(10, 3*len(layers)),
                       gridspec_kw={"hspace":1})
for l, layer in enumerate(layers):
    weights = layer.get_weights()
    for w, weight in enumerate(weights):
        tensor = layer.weights[w]
        ax[l, w].set_title(tensor.name)
        _ = ax[l, w].hist(weight[w].flatten(), 50)
```

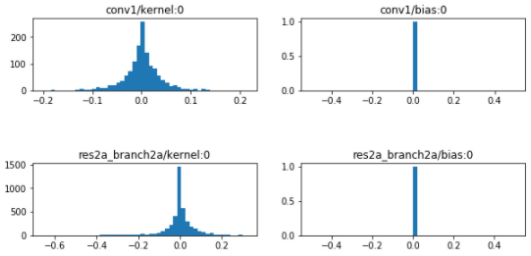


Figure 5. Histograms of weight and bias values for selected layers.

6. Results

Inference ran with batch size 1 and a detection threshold of 0.90. On validation image `image79.jpg` the scratch was localised with confidence above 92%. On the unseen image `test2.jpg` the model contoured the bumper scrape without false positives on the road, headlight or grille.



(a) Validation image (image79.jpg)



(b) Unseen test image (test2.jpg)

Figure 6. Predicted scratch masks and bounding boxes.

A colour-splash routine (`color_splash` in custom.py) keeps the detected damage in colour and renders the rest of the image in grayscale, which helps an insurance adjuster see the damage at a glance.

7. Discussion

Limitations	Recommended next steps
• Glare and reflections on metallic paint can mimic scratches at low thresholds. • Dirt, mud and water droplets resemble paint scrapes. • Scratches crossing panel seams are sometimes fragmented. • ResNet-101 is heavy; mobile use needs cloud offloading or quantisation.	• Split the single class into scratch, dent, panel tear and glass shatter. • Add monocular depth estimation (e.g. MiDaS) to measure dent depth. • Benchmark YOLOv8-seg or Mask2Former for low-latency edge deployment.

8. Conclusion

A Mask R-CNN model with a ResNet-101 + FPN backbone, fine-tuned from MS COCO weights on 68 annotated images, produced pixel-level damage masks on both validation and unseen test images. Weight diagnostics showed stable parameters, and polygon areas can be computed with the Shoelace formula to quantify damage. The results support instance segmentation for automated vehicle inspection and insurance workflows.

References

[1] K. He, G. Gkioxari, P. Dollár and R. Girshick, “Mask R-CNN,” in *Proc. IEEE ICCV*, 2017, pp. 2961–2969.

[2] S. Ren, K. He, R. Girshick and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE TPAMI*, vol. 39, no. 6, pp. 1137–1149, 2017.

[3] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan and S. Belongie, “Feature pyramid networks for object detection,” in *Proc. IEEE CVPR*, 2017, pp. 2117–2125.

[4] K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[5] A. Dutta and A. Zisserman, “The VIA annotation software for images, audio and video,” in *Proc. ACM Multimedia*, 2019.

[6] T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” in *Proc. ECCV*, 2014, pp. 740–755.